

Audit-Bericht: Dukatimer v0.5 - Umsetzungsstand

1. Architektur & Schichtenmodell (HAL / RTOS)

Feature	Status	Bewertung	Root Cause Analyse / Handlungsbedarf
Dual-Core Split	 OK	Exzellent	Core 0 (IO) und Core 1 (Realtime) sind physisch getrennt. Keine Blockaden mehr.
Input-Handling	 OK	Stabil	Interrupt-Migration abgeschlossen. <code>xInputQueue</code> ist die einzige Quelle der Wahrheit.
Display-Grounding	 OK	Gelöst	Shadow-Copy (<code>lcdShadow</code>) verhindert I2C-Jitter während der Belichtung.
Audio-Asynchronität	 OK	Gelöst	<code>SoundQueue</code> verhindert Blockaden durch <code>tone()</code> .
PSRAM-Integrität	 OK	Sicher	<code>paperBankPtr</code> liegt im PSRAM, Zugriffe sind via <code>gTimerMutex</code> geschützt.

2. Die "Root Cause" Engine (Physik & Mathe)

Feature	Status	Bewertung	Root Cause Analyse / Handlungsbedarf
Dosis-Berechnung	 Teilweise	Berechnet	Die Integration $D = E \cdot t$ läuft, aber die Auflösung ist noch zu grob.
Predictive Shutoff	 OFFEN	Kritisch	Lastenheft 5.2: Aktuell bricht der Task alle 10ms ab. Das erzeugt bis zu 10ms Jitter. Die geforderte Hardware-Timer-Interpolation fehlt im Code!
Guard Time Sequenz	 OK	Implementiert	Pre-Wait (200ms) und Post-Wait (400ms) sind fest im Timer-Life-Cycle verankert.
PaperBank Mathe	 Teilweise	Funktional	ISO-P und ISO-R Umrechnung (Punkt 7) ist im Code noch durch vereinfachte K-Werte ersetzt.

3. Modul-Status (Wizards & Logik)

Modul	Status	Feedback
Setup-Menü	 Final	Non-blocking, Queue-basiert, Thread-safe.
Preflash	 Final	Saubere State-Machine, exakte Millisekunden-Steuerung für Blitze.

Densitometer  Final I2C-Bus Schutz implementiert, Log10-Präzision auf `double` .

Calibration  Final Nutzt die asynchrone Spektral-Engine der Sonde.

4. Identifizierte Risiken & Defizite (Kritik)

A. Das Jitter-Trauma (Punkt 5.2 Lastenheft)

Dein Lastenheft fordert eine Genauigkeit von **< 1 ms**. Der aktuelle `vClosedLoopTask` in `Logic_Timer.cpp` pollt den Sensor alle 10ms.

- **Problem:** Wenn die Dosis zwischen zwei Polls erreicht wird, belichtet das System bis zu 9.9ms zu lang.
- **Verbesserung:** Wir müssen die im Lastenheft beschriebene **Interpolation** einbauen. Der Task muss berechnen, in wie vielen Mikrosekunden die Dosis voraussichtlich erreicht ist, und einen ESP32-Hardware-Timer (Alarm) für das Abschalten der LEDs setzen.

B. Die ISO-Dissoziation (Punkt 7 Lastenheft)

Du hast im Lastenheft ein komplexes Modell basierend auf ISO-P (Speed) und ISO-R (Range) beschrieben.

- **Problem:** In der `Logic_Math.cpp` und `Logic_Papers.cpp` arbeiten wir aktuell mit `Ksoft` und `Khard` . Das ist funktional, aber es ist noch nicht die "mathematische Eleganz", die das Lastenheft verspricht.
- **Verbesserung:** Implementierung einer Konvertierungsfunktion, die ISO-Werte in Lux-Sekunden Eckpunkte übersetzt.

C. Wireless-Telemetrie (C6 Probe)

Die Infrastruktur steht. Gemäß Workflow-Analyse ist die Sonde während der Belichtung inaktiv (keine Messung auf Papierebene vorgesehen).

- **Status:** Die Trennung der Funk-Wahrnehmung (C6) von der Belichtungs-Wahrnehmung (Head-Sensor) ist korrekt umgesetzt. Ein Stress-Test der Funkstrecke *während* der Belichtung ist hinfällig. Fokus liegt auf der Zuverlässigkeit des Handshakes *vor* dem Start.

5. Nächste Schritte (Priorisiert)

1. **Implementierung Predictive Shutoff:** Umbau der `Logic_Timer.cpp` , um einen Hardware-Timer (`esp_timer`) für das präzise Ende der Belichtung zu nutzen. Dies ist die "Wurzelbehandlung" gegen Zeitungenauigkeit.
2. **ISO-Mathe-Update:** Erweiterung der `Logic_Math.cpp` , um echte ISO-Datenblätter verarbeiten zu können (Umstellung von K-Werten auf Speed/Range Modell).
3. **Inbetriebnahme C6:** Finalisierung der `HW_Wireless.cpp` für den asynchronen Handshake im Idle-Zustand.